Does Call-Graph Fan-Out Predict Failure Impact in Microservices? A Preregistered Fault-Injection Study

DAKSHRAJSINH TANK[1] AND BADRINATH N[1]

[1]School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India
Corresponding author: [TO FILL: corresponding author name and email]

ABSTRACT One way to decide which services of a microservice system to harden first is to rank them by their position in the dependency graph, and an intuitive version of that ranking treats a service that calls many others as the riskiest. When a service fails, however, the services left waiting are its callers. This study tested whether the number of transitive callers of a service tracks how many services degrade when it fails, and whether a fan-out-weighted centrality metric does. A preregistered campaign ran 180 fault-injection runs on two DeathStarBench applications, giving each analysable service a process kill and an injected network delay in 5 repetitions. Dependency graphs were rebuilt from traces, latency was attributed to each service's own spans, and each degraded service was classified by its graph relation to the faulted one. On Social Network under kill faults, caller count and the number of degraded callers correlated at $\rho = 0.785$, but none of the 44 confirmatory tests survived Benjamini-Hochberg correction (adjusted $p = 0.3451$), and a latency-fault replication agreed. Much of the correlation is definitional, since ten of the eleven services degraded every caller they had. The fan-out metric correlated negatively in all four architecture-by-fault cells and was distinguishable from zero in none; a post hoc sweep of 231 weightings found no significant association with degraded callers. Degradation reached callers and almost never callees: no descendant of the faulted service degraded in 179 of 180 runs.

INDEX TERMS Distributed tracing, failure propagation, fault injection, graph centrality, microservices, preregistration

I. INTRODUCTION

A microservice application is built from many small services that call one another over the network [1]. Each service can be changed and deployed on its own, which is much of the appeal. The price is that a fault in one service can show up far from where it started, and industrial experience suggests such faults are slow to diagnose [2]. Teams that run these systems therefore want to know, before anything breaks, which services to harden first. Injecting faults into every service to find out is expensive. A cheaper option is to read the answer off the dependency graph.

Graph measures have a long record as indicators of importance and risk. The standard notions of centrality were set out for social networks [3], and network measures computed on software dependency graphs have been used to predict which components will have defects [4]. Applied to a service graph, the usual intuition is that a service which calls many others is a hub, and that hubs are dangerous. This project began from that intuition. Its original metric gave extra weight to fan-out, the number of services a service calls directly, on the reasoning that a service with many callees cuts many connections when it fails.

A simple argument points the other way. When a service fails, the services that wait on its answers are the ones that call it. Its callees lose a client and see less traffic. If that argument holds, the number of services that degrade should follow a service's transitive callers, its ancestors in the graph, and a fan-out ranking should go wrong precisely for the services it rates highest. Social Network's composition service shows the conflict in one example. It calls 7 services directly, and the fan-out metric ranks it first of the 11 services analysed. It has a single caller, and by caller count it ranks last.

This study tested the caller-based prediction, with the fan-out metric alongside it, in a preregistered fault-injection campaign [5] on two applications from the DeathStarBench suite [6]. The caller-based hypothesis was formulated after a pilot study and locked, together with every analysis choice, before any campaign data existed; the fan-out metric was kept as a comparator. The campaign completed all 180 planned runs. Every analysable service received both fault types, a process kill and an injected network delay, with 5 repetitions per cell. Each dependency graph was rebuilt from traces, latency was attributed to the spans each service emits itself, and every degraded service was classified by its position relative to the faulted one, so that degradation of callers and of callees could be counted apart.

The preregistered hypothesis was not confirmed. On Social Network under kill faults, caller count and the number of degraded callers correlated at $\rho = 0.785$, yet none of the 44 tests in the confirmatory family survived correction, and the replication under latency faults gave the same answer. Much of that correlation is definitional, because the outcome counts a subset of the predictor and most services degraded every caller they had. The fan-out metric did no better. Its correlation with degraded callers was negative in every architecture and fault type and was never distinguishable from zero. The plainest result needs no test at all: degradation reached callers and almost never callees, with no descendant of the faulted service degraded in 179 of 180 runs.

The work cited above leaves a specific question open. Network measures on software dependency graphs have been shown to predict where defects accumulate [4], but a defect count describes the code, not how far a run-time failure spreads under load. Fault-injection practice and tools [7]–[9] are built to check that a system tolerates failure or to find bugs; they are not used to test whether a structural ranking anticipates the damage a failure does. Root-cause localisation [2], [10] uses the service graph together with live metrics after an anomaly has begun, which is a different task from ranking services before any fault occurs. Trace-derived call graphs also change with the traffic that produces them [11], so a graph-based ranking inherits that dependence. None of the cited works reports a test, fixed in advance, of a graph-derived

ranking against failure propagation measured in a controlled campaign.

This paper makes four contributions. First, it reports a preregistered test of whether caller count, or a fan-out-weighted centrality metric, tracks measured failure impact in two benchmark applications, with its null result and with the power ceiling the design imposed. Second, it documents a measurement pitfall, in which a request's end-to-end latency is charged to every service on its path, and its effect in the pilot, where it flagged services unrelated to the faulted one in the graph. Third, it reports an observation on the direction of propagation that held in both applications and under both fault types. Finally, it provides an artifact in which every number in the paper is produced by a script from logged campaign data.

Section II reviews related work. Section III states the problem and the hypotheses, and Section IV describes the testbed and the methodology. Section V reports the results and Section VI discusses them. Section VII sets out the threats to validity, and Section VIII concludes.

II. RELATED WORK

A. Graph Measures as Risk Indicators

Centrality has well-established definitions. Freeman's treatment separated several distinct ideas of what makes a node central, some based on a node's direct ties and some on the shortest paths that pass through it, and gave them normalised forms that are still in use [3]. Path-based measures were expensive to compute on large graphs until Brandes published a fast algorithm for betweenness [12]. NetworkX 3.6.1 (https://networkx.org), the graph library used in this study [13], implements that algorithm.

Software engineering adopted these measures early. Zimmermann and Nagappan computed network measures on the dependency graph of a software system and found that they helped predict which components would contain defects [4]. That work and the present study share a premise, that a component's position in a dependency graph says something about its risk. The outcomes differ. A defect count describes where bugs accumulate over a release, while this study asks how far a run-time failure spreads once it has happened, which is a property of the call pattern under load and not of the code alone.

B. Fault Injection and Chaos Engineering

Deliberately injecting faults into a running system is now a standard way to learn how it behaves under stress. Basiri and colleagues describe the practice as it developed at Netflix, where engineers run controlled experiments on production systems to build confidence that they tolerate failure [7]. For microservices specifically, Gremlin offered a framework for systematic resilience testing, injecting faults between services and checking whether the application handles them as intended [8]. Lineage-driven fault injection takes a different route. It reasons backwards from how a correct result was produced to choose the faults most likely to expose a bug, so that fewer experiments are wasted [9].

The use of fault injection in this study is narrower than any of these. It does not search for bugs or validate fallbacks. Two fault types are injected into each service in turn, in a fixed and balanced design, and the counts of degraded services serve as the outcome of a structural prediction. The faults are a measuring instrument here.

C. Call Graphs from Traces

Distributed tracing makes it possible to recover a service graph from what the system actually does. Dapper established the model most tracing systems still follow: a request carries a trace context across service boundaries, and each unit of work is recorded as a span with a parent, so a request becomes a tree of spans [14]. Reading parent and child spans from different services gives a caller-callee edge, which is how the graphs in this study were built.

Graphs built this way depend on the traffic that produced them. Luo and colleagues analysed call graphs from Alibaba's production traces and found them to be highly dynamic, changing from one request to the next [11]. The benchmark graphs here are small and the workload is fixed, so far less variation appears, but the same dependence on traffic affects the predictor, as Section VII-F explains. The observed Hotel Reservation graph already differs from the one the benchmark documents.

D. Root-Cause Localisation

A large body of work starts from the other end of the problem. Given an anomaly that is already happening, root-cause localisation tries to find the service responsible. MicroRCA is one example. It builds a graph of the running system and ranks candidate services by how their behaviour relates to the observed performance problem [10]. An industrial survey by Zhou and colleagues documents how hard such diagnosis is in practice, and contributes a benchmark system with injected faults for studying it [2].

These methods use the graph after the fact, together with live metrics. The question here comes earlier: whether graph structure alone, before any fault occurs, indicates which services will do the most damage when they fail. The two problems share the dependency graph but ask different things of it. The propagation direction reported in Section V-F may still be of interest to diagnosis work, though no diagnosis method was studied.

E. Preregistration and Multiple Testing

Preregistration separates the analyses planned before data are seen from those devised afterwards, so that a reader can tell a test of a prediction from a search through the data [5]. Every hypothesis and analysis choice in this study was locked before the campaign, every later departure was logged, and each post hoc analysis is labelled as such. With 44 tests in each family, some correction for multiple testing was unavoidable. The false discovery rate is controlled with the Benjamini-Hochberg procedure [15], applied separately within each family, which keeps the expected proportion of false positives among reported discoveries at the chosen level while giving up less power than a family-wise correction would.

III. PROBLEM STATEMENT AND HYPOTHESES

A. Call Graph, Faults and Outcome

Let $G = (V, E)$ be the directed call graph of an application, where V is the set of services and an edge $(u, v) \in E$ means that u calls v under the traced workload, and let $u \rightsquigarrow v$ denote a directed path from u to v in G. The ancestors of a service are the services from which it can be reached, and its descendants are the services it can reach:

$$\mathrm{anc}(v) = \{u \in V : u \rightsquigarrow v, u \neq v\} \quad (1)$$

$$\mathrm{desc}(v) = \{u \in V : v \rightsquigarrow u, u \neq v\} \quad (2)$$

A fault $f \in$ {kill, latency} applied to service v either terminates its process or delays its network traffic (Section IV-F). In repetition r, the degraded set $D_r(v, f)$ contains the services other than v whose own latency or error rate crosses the degradation rule of Section IV-H during the fault window. The degraded set is split into three disjoint parts by graph relation to the faulted service:

$$D_r^A = D_r \cap \mathrm{anc}(v),\ D_r^D = D_r \cap \mathrm{desc}(v) \quad (3)$$

$$D_r^U = D_r \setminus (\mathrm{anc}(v) \cup \mathrm{desc}(v)) \quad (4)$$

The primary outcome of a service is the median over its repetitions of the number of degraded ancestors, $Y(v, f) = \mathrm{median}_r\, |D_r^A(v, f)|$, recorded as `ancestor_affected_count`. The counts of degraded descendants and of degraded unrelated services are recorded separately and are never added to it, because a combined count would discard the direction in which a failure travels.

B. Predictors

The primary predictor is the number of transitive callers, $X(v) = |\mathrm{anc}(v)|$, recorded as `ancestor_count`. The comparator of interest is the hybrid criticality metric of the original proposal:

$$\mathrm{HC}(v) = w_{in} C_{in}(v) + w_{out} C_{out}(v) \varphi(v) + w_{btw} C_{btw}(v) \quad (5)$$

$$\varphi(v) = d^+(v) / \max_u d^+(u) \quad (6)$$

Here C_{in} and C_{out} are in-degree and out-degree centrality, C_{btw} is betweenness with directed normalisation, d^+ is out-degree, and the weights are $w_{in} = 0.4$, $w_{out} = 0.4$ and $w_{btw} = 0.2$ (Section IV-D). The fan-out weight φ is linear in out-degree, so the metric rewards services that call many others. Ten further graph measures are kept as comparators. Each test pairs one predictor with one outcome across the services of one application, with the gateway excluded (Section IV-C).

C. Hypotheses

The hypotheses are those of the locked preregistration. H2 below carries the label H1b in that document, where the name H2 belongs to a structural observation that is not tested here.

H1, the primary hypothesis, states that Y is predicted by X, the number of transitive callers, and not by out-degree or descendant count, nor by the hybrid metric. It is tested as a positive Spearman rank correlation between X and Y:

$$\mathrm{H1}: \rho(X, Y) > 0 \quad (7)$$

H2 is directional. It states that the fan-out premise points the wrong way, so that the hybrid metric is not positively associated with the outcome:

$$\mathrm{H2}: \rho(\mathrm{HC}, Y) \leq 0 \quad (8)$$

Under the null hypothesis of each test there is no monotone association between predictor and outcome. Every p-value is two-sided and comes from a permutation test, and it is corrected with the Benjamini-Hochberg procedure within its family (Section IV-I). Kill faults form the confirmatory family, and latency faults repeat every test as a separate replication family.

IV. TESTBED AND METHODOLOGY

Figure 1 shows the pipeline in the order the campaign executed it. The six steps in the first column built the dependency graphs and the predictors and locked the analysis plan before any campaign data existed. The next four repeated in every run, and the last two ran once the campaign was complete. Each box names the repository files that carry out the step.

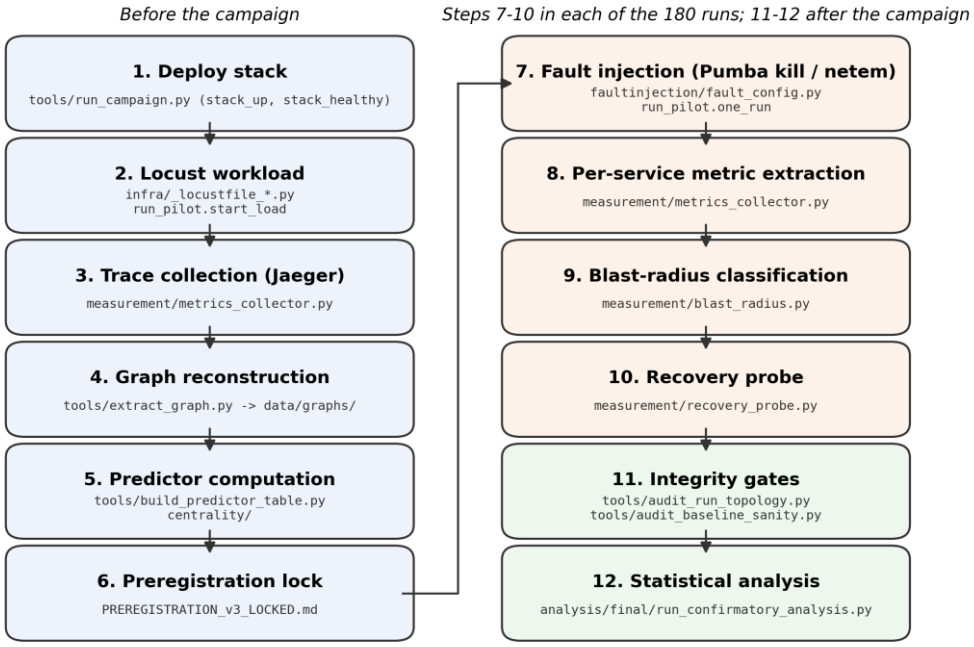


Fig. 1. Methodology pipeline in execution order; each box names the repository files that carry out the step, and the steps from fault injection to the recovery probe repeat for every run.

A. Testbed

All experiments ran on a single host whose configuration is given in Table I. The application stacks, Jaeger and Pumba ran as containers inside the virtual machine of Docker Desktop's WSL2 backend, while the load generator and the campaign runner ran as host processes (Fig. 2). The runner stops every stack before the campaign and brings up one application at a time, stopping it before the other starts, so the two never shared the 7.598 GiB memory ceiling of the virtual machine. The Social Network stack has 27 containers and the Hotel Reservation stack 24, as recorded before the campaign. Traces were collected with Jaeger 1.62.0 (https://www.jaegertracing.io), one all-in-one instance per application with in-memory storage.

TABLE I
TESTBED AND RUN PROTOCOL. HOST VALUES MARKED MEASURED WERE QUERIED AFTER THE CAMPAIGN; ALL OTHERS WERE RECORDED BEFORE OR DURING IT

item	value	source
Host OS	Microsoft Windows 11 Home Single Language (10.0.26300)	measured on 2026-10-20
Host CPU	12th Gen Intel(R) Core(TM) i7-1255U, 10 cores, 12 threads	measured on 2026-10-20
Host RAM	15.7 GB	recorded 2026-09-24
Docker Desktop	4.91.0, WSL2 backend	version measured on 2026-10-20; backend recorded 2026-09-24
Docker VM memory	7.598 GiB (WSL2 default)	recorded 2026-09-24
Docker client / Compose	29.8.0 / 5.5.1	client recorded 2026-10-18; Compose measured on 2026-10-20
Containers per stack	Social Network 27, Hotel Reservation 24	recorded 2026-09-24; one stack ran at a time
DeathStarBench commit	6ecb097	pinned upstream commit
Jaeger	1.62.0 all-in-one, in-memory storage	compose override
Pumba	ghcr.io/alexei-led/pumba (untagged image)	faultinjection/fault_config.py; version not recorded
Python	3.13.2	README
Python libraries	NumPy 2.1.2, pandas 2.2.3, SciPy 1.14.1, statsmodels 0.14.6, NetworkX 3.6.1, matplotlib 3.9.2	README
Locust	2.46.3; 25 users, spawn rate 10 users/s, wait 0.5-1.5 s between requests	runner and locustfiles
Request mix, Social Network	compose post 3, home timeline 5, user timeline 2	task weights
Request mix, Hotel Reservation	search 6, reservation 2, user 2, recommendation 1	task weights
Load before the fault (s)	60; the baseline is computed over this window	runner
Fault hold (s)	30	runner
Recovery probe (s)	poll 1, confirmation 5, window lag 5, cap 180	runner
Cooldown (s)	20	runner

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

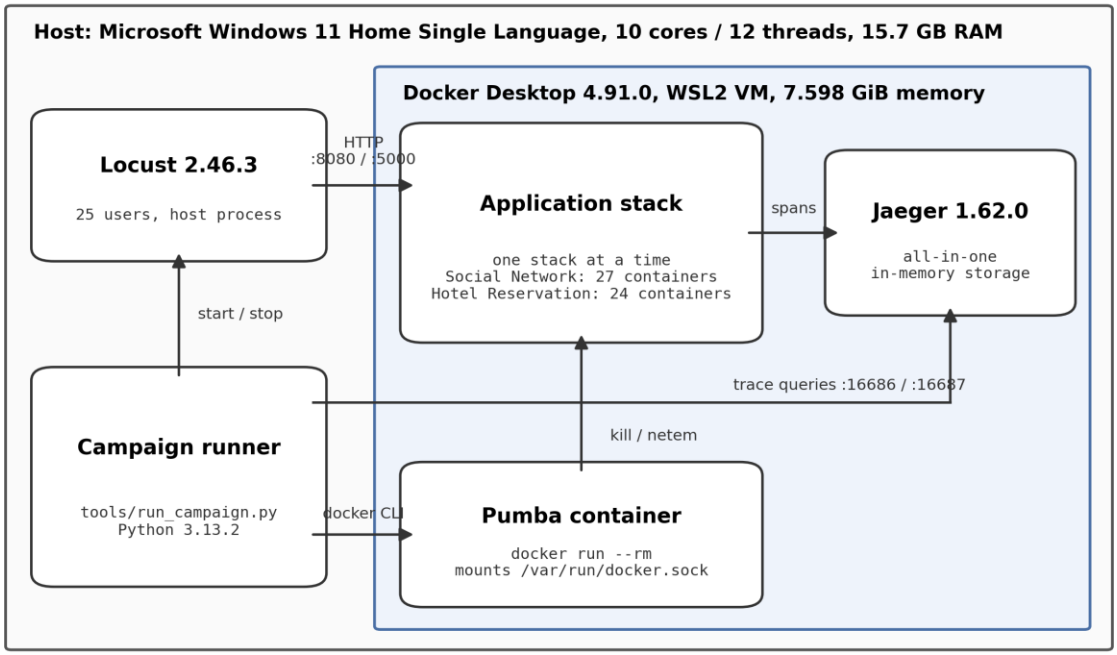


Fig. 2. Testbed deployment: the host, the Docker Desktop VM, the application stack, Jaeger and Pumba containers, and the host-side Locust and campaign runner.

The host operating system, CPU and Docker Desktop version in Table I were queried on 2026-10-20, after the campaign, and are labelled as measured. The memory ceiling, host memory and container counts were recorded on 2026-09-24, before the campaign. Values with no record from the campaign period are omitted.

B. Applications and Dependency Graphs

Both applications come from DeathStarBench, Social Network and Hotel Reservation [6], built from one pinned upstream commit, `6ecb097`. The code under test was therefore the same in every run. The two applications have different shapes. Social Network is deep and fans out from a single composition service, while Hotel Reservation is shallow and close to a star around its gateway. That difference ended up deciding what each application could show at all (Section V-B).

The dependency graph was not taken from documentation. A diagram shows the calls a system can make, while the hypotheses concern the calls it does make under the workload being perturbed. Each graph was therefore rebuilt from distributed traces, with a directed edge from caller to callee wherever a span from one service is the parent of a span from another [14]. The Social Network graph came from 2,449 traces holding 32,847 spans, and the Hotel Reservation graph from 3,446 traces holding 31,155 spans. The graphs have 12 and 8 service nodes, with 17 and 7 observed call edges, and Fig. 3 and Fig. 4 show them as observed. Both are frozen as files, and every predictor is computed from those files, so no later change of workload can shift a structural value.

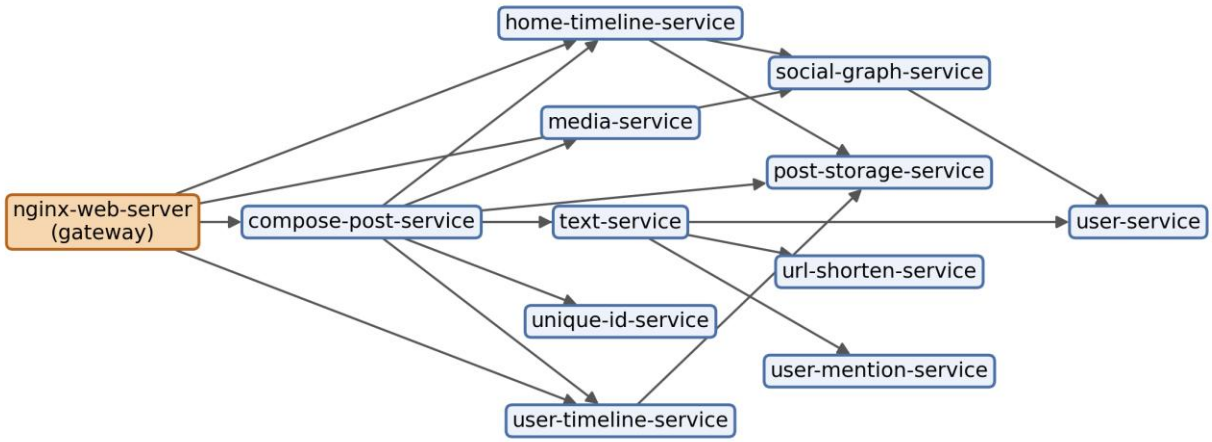


Fig. 3. Social Network dependency graph as observed in traces: 12 services and 17 caller-to-callee edges, gateway marked.

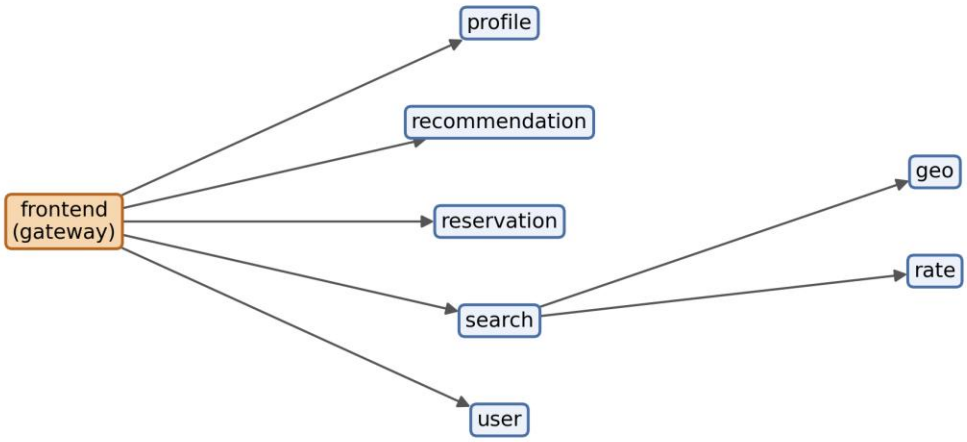


Fig. 4. Hotel Reservation dependency graph as observed in traces: 8 services and 7 caller-to-callee edges, gateway marked.

The observed Hotel Reservation graph differs from the documented one. Of the 8 call edges implied by the published service tree, the traces recover 6. Two never appear under this workload: `reservation→user` and `search→profile`. One edge appears that the tree does not list, `frontend→profile`, which appears to stand in for the missing `search→profile`, with the gateway fetching profiles directly. The difference matters for one service. `user` has a single observed caller, `frontend`, where the documentation implies two, so its ancestor count is 1. With the documented graph, `user` would carry a predictor value that no traffic in the experiments could have produced. The observed graph is kept and the divergence reported.

C. Analysis Set and Gateway Exclusion

Every correlation excludes the one service in each application that has no caller in the graph: `nginx-web-server` in Social Network and `frontend` in Hotel Reservation. A gateway receives traffic only from the load generator, which is not a node in the graph, so its ancestor count is zero by construction and cannot vary. Killing it also removes the only way into the application, which leaves the outcome undefined. Excluding the gateways leaves 11 services in Social Network and 7 in Hotel Reservation. In Hotel Reservation it also removes the service with the highest fan-out, a scope limit taken up in Section VII.

D. Predictors

The primary predictor and the hybrid metric are defined in Section III-B. The ten further measures kept as comparators are degree, in-degree, out-degree, betweenness, closeness, eigenvector centrality, PageRank, a composite rank, descendant count and dominator subtree size. The plan forbade dropping any of them once results were seen, and

none was dropped. Centralities follow the standard definitions [3], and betweenness is computed with the algorithm of Brandes as implemented in NetworkX [12], [13].

The hybrid weights of 0.4, 0.4 and 0.2 come from the project's original proposal, in which an empirical search chose them; they were therefore not fixed independently of all data. The locked plan called for tuning the weights on Social Network and freezing them for Hotel Reservation, and that step was never run (Deviation 10). No re-tuning on campaign data took place. A sensitivity analysis over every weighting on the preregistered grid, run after the confirmatory results were known, is reported in Section V-J.

Two predictor-architecture pairs were declared degenerate in advance, because the predictor is constant across the analysis set and Spearman's ρ is then undefined. Eigenvector centrality is 0 for every analysed Social Network service, the reconstructed graph being acyclic, and every Hotel Reservation service has the same in-degree, each having exactly one caller. Both pairs are left out of the test family. A degenerate graph property is not a null result, and entering it as one would dilute the false-discovery-rate correction. That leaves 22 testable pairs of the 24 considered.

E. Workload and Run Protocol

Each run starts the load generator, Locust 2.46.3 (https://locust.io), with 25 users spawned at 10 per second. Each simulated user waits between 0.5 and 1.5 seconds between requests, and each request is drawn from a fixed weighted mix of tasks. On Social Network the weights are 3 for composing a post, 5 for reading a home timeline and 2 for reading a user timeline. On Hotel Reservation they are 6 for a hotel search, 2 for a reservation, 2 for a user lookup and 1 for recommendations.

The load runs for 60 seconds before the fault, and the fault-free baseline percentiles are computed over that same 60-second window. The window therefore begins when the load generator starts and includes its ramp-up; its length was identical in all 180 runs. The fault is then injected and held for 30 seconds, the recovery probe runs, and a 20-second cooldown separates one run from the next.

The design is fully crossed and balanced. Each analysable service receives each fault type in 5 repetitions, giving 36 cells and 180 runs, of which 110 are Social Network runs and 70 Hotel Reservation runs. A run whose telemetry is incomplete is discarded and repeated in the same cell; it is never topped up elsewhere and never analysed as zeros. The same rule covered the 17 runs invalidated by deviations 1, 2 and 4 (3, 3 and 11 runs; Section IV-J), which were collected again in their own cells. Measured from the first to the last end-timestamp of the kept runs, the campaign ran from 2026-09-26 05:02 to 13:35 UTC, 8.5 hours in all, and lost 0 runs to telemetry failure. Earlier records give 9.3 hours because they also count the 3 pre-launch rehearsal runs.

The raw spans of both the baseline and the fault window were kept for every run. Storing them is cheap, and it means any later change to a metric definition can be replayed against the original data instead of being re-run. Several analyses in this paper depend on that.

F. Fault Injection with Pumba

Faults are injected with Pumba (https://github.com/alexei-led/pumba), which the runner starts as a short-lived container (image `ghcr.io/alexei-led/pumba`, with no version tag) that acts on the target container through the Docker socket. The runner issues every command through the Docker 29.8.0 client (https://www.docker.com). Table II lists the command lines, generated from the runner code. A kill fault sends SIGKILL, which ends the process without letting it drain connections or answer requests in flight. A latency fault applies a `tc netem` (https://wiki.linuxfoundation.org/networking/netem) rule that adds 500 ms of delay with 100 ms of jitter to the container's network interface for 30 seconds. The kill asks what happens when a dependency vanishes; the delay asks what happens when it stays up but stops being timely.

TABLE II
COMMAND LINES ISSUED FOR EACH RUN, GENERATED FROM THE RUNNER CODE (THE RUNNER STARTS LOAD, RESOLVES THE TARGET CONTAINER, INJECTS THE FAULT AND RESTORES THE SERVICE)

step	command	source
Load generation	`python -m locust -f <locustfile> --host <url> --headless --users <users> --spawn-rate 10 --run-time <duration>s --only-summary --loglevel ERROR`	tools/run_pilot.py: start_load
Target resolution	`docker ps --filter label=com.docker.compose.service=<service> --format '{{.Names}}\t{{.Label "com.docker.compose.service"}}'`	tools/run_pilot.py: container_for
Kill fault	`docker run --rm -v /var/run/docker.sock:/var/run/docker.sock ghcr.io/alexei-led/pumba kill --signal SIGKILL <container>`	faultinjection/fault_config.py: pumba_command
Latency fault	`docker run --rm -v /var/run/docker.sock:/var/run/docker.sock ghcr.io/alexei-led/pumba netem --duration 30s delay --time 500 --jitter 100 <container>`	faultinjection/fault_config.py: pumba_command
Restore after a kill	`docker compose up -d <compose_svc>`	tools/run_pilot.py: one_run
Clean-up after a latency fault	`docker compose up -d --force-recreate <compose_svc>`	tools/run_pilot.py: one_run

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

To find the target container, the runner looks for the one whose `com.docker.compose.service` label, which Docker Compose sets on every container, equals the service name exactly. The runner stops if no running container or more than one matches, so a fault cannot land on a different service from the one recorded (Deviation 2). The Social Network gateway reports itself to Jaeger as `nginx-web-server` but runs as the compose service `nginx-thrift`, and the runner translates between the two names.

Timing differs by fault type. A kill completes at once; the runner waits out the fault window and then restores the service with `docker compose up -d`, and the fault-removal instant

is taken when that command returns. Because `pumba netem` blocks for its whole declared duration, the runner launches it as a separate process, waits out the window, and takes the fault-removal instant when that process exits. Running it synchronously would let the rule expire before the measurement window had opened. After the recovery probe of every non-kill run, the runner recreates the target container with `--force-recreate`, so that no residual netem rule can reach the next run (Deviation 4). Algorithm 1 gives one run in full, with the warm-up and baseline window W = 60 s, the fault hold H = 30 s, probe interval P = 1 s, confirmation period K = 5 s, probe lag L = 5 s and recovery cap C = 180 s.

Algorithm 1: One campaign run (tools/run_pilot.py, one_run), called in manifest order with a cooldown after each run

Input: application a, service s, fault type f, repetition r; graph G of a
c ← the running container whose label com.docker.compose.service equals the compose name of s; stop unless exactly one matches
start the Locust load against the gateway of a
wait W; B ← per-service baseline percentiles and error rates over the last W
t_inj ← now; issue the Pumba command for f on c ▷ kill: synchronous; latency: background process
wait H; t_end ← now
Dg ← services degraded in [t_inj, t_end] relative to B (Section IV-H)
persist the raw spans of the baseline and fault windows
if f = kill then run docker compose up -d for s else wait for the Pumba process to exit
t_rm ← now ▷ fault-removal instant
T_rec ← recovery probe from t_rm with interval P, confirmation K, lag L and cap C
if f ≠ kill then run docker compose up -d --force-recreate for s
classify Dg into ancestors, descendants and unrelated services of s in G
stop the load; return the run record

G. Per-Service Latency and Error Attribution

The latency definition needed more care than any other part of the method, because the obvious implementation is wrong in a way that is easy to miss.

A distributed trace records a whole request path. A tracing backend asked for the traces involving X hands back every trace in which X appears, and each of those traces has a root span whose duration is the end-to-end latency of the whole request. A percentile taken over those root spans and called "the p95 of X" is really the end-to-end p95 of all requests that passed through X, and every service on a request path receives the same value. Error rate has the same flaw if a trace counts as failed whenever any span in it carries an error.

Services on a shared path then become indistinguishable, and any threshold applied to them flags them as a group. Figure 5 shows the effect on Hotel Reservation under a latency fault injected into `search`. The figure uses two pilot method-validation runs, which are not among the 180 campaign runs, and its end-to-end values are reconstructed from their persisted spans. Under that scope three services, `geo`, `rate` and `search`, share an identical baseline p95 in both runs, and a multiplicative threshold flags five services that were not faulted. Two of these, `profile` and `reservation`, are neither ancestors nor descendants of `search`. They share a trace with it and nothing more.

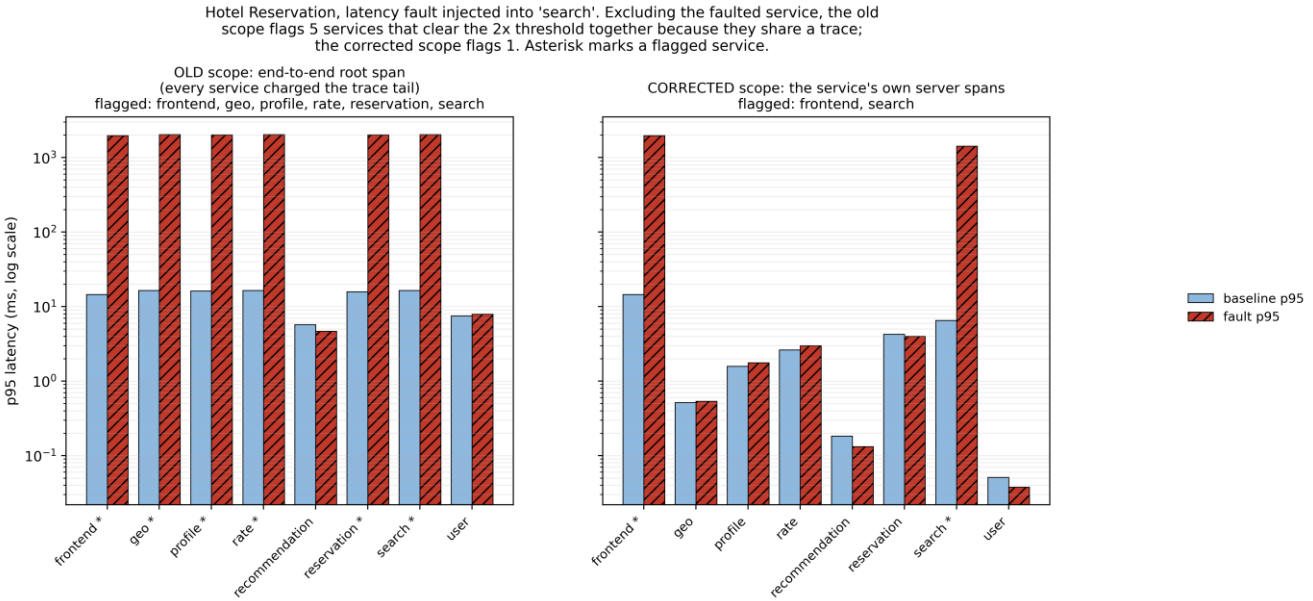


Fig. 5. Per-service p95 under the old end-to-end root-span scope vs the corrected own-span scope, HR search latency fault.

Each service's latency is therefore measured on the server spans that service emits. On Hotel Reservation, whose spans carry a span-kind tag, client spans are excluded, since a client span recorded by X times the callee as X sees it and would pass a downstream delay back to X. Social Network's spans carry no span-kind tag, so for Social Network all of a service's spans were used. That applied to 2,598 of 2,598 Social Network samples, and about half of the spans used, 49.7%, are client spans. The run records nonetheless label these samples as server spans. A post hoc replay limited to server spans altered the outcome of 2 of the 110 runs and none of the service-level medians, and the primary result came out identical (Deviation 11). An error counts against a service only when one of that service's own spans is tagged as an error, and the rate is divided by the number of traces in which the service appears. End-to-end root-span percentiles are kept

only for the gateway, where they are the right quantity, and every returned record carries a scope tag that says which definition produced it.

With per-service attribution, the same Hotel Reservation runs flag one non-faulted service instead of five, and that service is an ancestor of the faulted one. The services that drop out do more than fall below the threshold. Their own service time stays flat, and in one case drops slightly, while the end-to-end value previously assigned to them grows more than a hundredfold. Contention for shared resources would be expected to raise a service's own service time, so this pattern points to a measurement artifact.

H. Outcomes: Blast Radius and Recovery Time

For each run, the services degraded during the fault window are found and each is classified by its graph relation to the faulted node. A service counts as degraded when either of two conditions holds on its own spans. The latency condition requires its fault-window p95 to exceed 2.0× its baseline p95 and to rise by at least 1.0 ms. The error condition requires its error rate to rise by at least 0.05 in absolute terms.

The absolute floor became necessary once percentiles were truly per-service. Some services here have baseline p95 values in the hundredths of a millisecond, and at that scale a purely multiplicative rule reacts to scheduler noise. A service whose p95 rises threefold between two consecutive fault-free windows passes a 2× test without any physically meaningful change. Both conditions are evaluated at full floating-point precision, and values are rounded only for display, since rounding to two decimals first would leave a step about as large as the whole signal of the fastest services. The run executor and the recovery probe share a single implementation of the latency rule, which the offline replay tool also calls. The error-rate rule is repeated in three modules with the same value.

Degraded services are split into the three counts of Section III-A, which are recorded and analysed separately. `ancestor_affected_count`, the primary outcome, counts degraded transitive callers of the faulted service. `descendant_affected_count` counts degraded transitive callees. `unrelated_affected_count` counts degraded services that are neither, which would indicate shared-resource contention or synchronous fan-out.

Which indicator carries signal depends on the fault type, and the pairing was fixed in the plan. Under kill faults the primary indicator is error rate, and Δp95 is not used. In the pilot, across 10 kill runs, the sign of kill-fault Δp95 split by application. It was negative in every Hotel Reservation run, where p95 fell even as most requests errored, and positive in every Social Network run. The likely explanation, that Hotel Reservation's services fail requests quickly when a dependency dies while Social Network's clients wait for a timeout, was not tested. Kill-fault Δp95 is never pooled across architectures. Under latency faults the roles reverse: Δp95 is the primary indicator, and error rate carries no signal because nothing has failed. Each fault type is therefore analysed with its own preregistered indicator.

Recovery time, T_{rec}, is the number of seconds from fault *removal* until Δp95, Δp99, Δerror-rate and the downstream-affected count are all back within 10% of their pre-fault baseline and stay there for a 5-second confirmation period. Timing from removal matters. Timed from injection, T_{rec} would include the whole fault duration and could never fall below it. The recovery probe polls at a nominal 1-second interval. Measured as the median gap between successive samples within each run, its effective resolution was 1.4 to 2.4 seconds. Each sample is drawn from a window lagged 5 seconds into the past, because tracing backends flush spans in batches and a window ending at the present instant under-counts. A sample needs at least 20 traces before its percentiles can vote for recovery, so a handful of fast requests cannot end a run early. The tolerance is one-sided, and an empty or thin window counts as no signal. No probe window may reach back past the fault instant.

A run that has not recovered within 180 seconds is recorded as right-censored, with the observation horizon stored beside it, and is never filled in with a number. In the final dataset 0 of the 180 runs were censored. Observed recovery times ranged from 9.72 s to 156.27 s, with a median of 15.74 s. The plan stated in advance that if fewer than 10% of observations were censored, the rank test would become the primary inferential result for T_{rec} and survival models would be reported beside it. That rule fired on the data as collected.

I. Statistical Analysis

Associations between a graph predictor and a measured outcome are tested with Spearman's ρ at the service level, after taking the median over each service's repetitions. Significance comes from permutation tests, exact where the permutations can be enumerated and otherwise from 10,000 random permutations. The asymptotic approximation is unreliable at these sample sizes and with the ties these graphs produce. Confidence intervals are bias-corrected and accelerated bootstrap intervals, with a percentile interval used, and disclosed, where the ties leave the bias correction undefined. The analysis was developed and verified on Python 3.13.2 (https://www.python.org), with SciPy 1.14.1 (https://scipy.org) supplying ranks and the reference Spearman implementation, and statsmodels 0.14.6 (https://www.statsmodels.org) the Cox and log-rank models.

Tests fall into three families, each corrected separately with the Benjamini-Hochberg procedure at q = 0.05 [15]. The confirmatory family holds 44 tests on kill faults. The replication family holds 44 tests on latency faults and is reported as a replication. A third, exploratory family covers secondary outcomes and carries no confirmatory weight. The locked plan considered pooling all families into one correction and rejected it, on the grounds that the confirmatory and replication families ask the same question under different fault types and should not dilute each other. For the primary outcome, the replication in fact reuses the same outcome values as the confirmatory family (Section V-E).

Two gates run before any statistic is computed. One confirms that every expected service was present in every run. The other checks that each run's fault-free baseline is a

plausible steady state, flagging any baseline that is an outlier against the rest of the campaign by median and median absolute deviation. Both must report zero exceptions before analysis can proceed. All 10 pre-analysis integrity checks passed on the final dataset.

J. Preregistration and Deviations

The plan was specified and locked before the campaign started [5], and every later change was logged with its date and with whether campaign data had been analysed by then. Twelve deviations were recorded, and the Appendix lists them. Nine invalidated no data. The other three invalidated 17 runs in all, each collected again in its own cell, so the final dataset is complete and balanced. Deviation 1 arose in the pre-launch rehearsal. Per-service percentiles of a few hundredths of a millisecond made a purely multiplicative degradation rule react to scheduler noise, so the absolute floor of Section IV-H was added and the 3 rehearsal runs were discarded. Deviation 2 was a targeting defect in which a substring match faulted the wrong Hotel Reservation container; exact label matching replaced it, and 3 runs were collected again. Deviation 4 was a netem rule that outlived its run and inflated the baselines of later latency runs; the 11 affected runs were collected again, and the per-run container recreation and a baseline-sanity gate were added. Deviation 3 was a name mismatch in the new stack check that stopped the campaign on a healthy stack. Deviations 5, 6, 7, 8 and 9 concern the analysis: two choices the plan left open, the rank test becoming primary for recovery time because no run was censored, an asymptotic p-value quoted in the plan, statistical routines vectorised for speed and tested against the reference implementation, and exact permutation p-values computed after the results were known and reported only as supplementary. Deviation 10 is the weight tuning that was not run (Section IV-D), Deviation 11 concerns Social Network span selection (Section IV-G), and Deviation 12 records a latency run attempted twice, of which only the completed second attempt is analysed.

K. Data and Code Availability

The code, the raw campaign data (persisted trace spans, run records and the reconstructed dependency graphs) and the analysis outputs are available at https://github.com/Daxx133-bot/proj-1-updated and will be archived at Zenodo (DOI: [TO FILL]). The repository's build script regenerates every number in this paper from the logged data. The experiments used DeathStarBench at upstream commit 6ecb097. Code is released under the MIT licence; data and figures are released under CC BY 4.0.

V. RESULTS

A. Campaign Completion

All 180 planned runs completed, across 36 balanced cells. No run was lost to telemetry failure, no gateway was faulted, and every recovery time was observed. Table III and Table IV list the graph predictors and measured outcomes for each service in Social Network and Hotel Reservation.

TABLE III
SOCIAL NETWORK: GRAPH PREDICTORS PER SERVICE WITH ANCESTOR_COUNT AND THE MEASURED SERVICE-MEDIAN OUTCOMES

service	ancestor_ count (services)	descendant_ count (services)	in_degree_ raw (callers)	out_ degree_ raw (callees)	hybrid_ criticality	betweenness	closeness	pagerank	eigenvector	measured_ ancestor_ affected_ kill (services)	measured_ ancestor_ affected_ latency (services)	T_rec_kill_ s (s)	T_rec_ latency_s (s)
post-storage-service	4	0	3	0	0.109	0.000	0.291	0.157	0.000	4.0	4.0	15.31	15.45
user-service	4	0	2	0	0.073	0.000	0.242	0.145	0.000	2.0	2.0	14.52	14.83
social-graph-service	3	1	2	1	0.081	0.014	0.205	0.098	0.000	3.0	3.0	14.99	15.42
url-shorten-service	3	0	1	0	0.036	0.000	0.136	0.080	0.000	3.0	3.0	16.05	15.23
user-mention-service	3	0	1	0	0.036	0.000	0.136	0.080	0.000	3.0	3.0	14.51	14.45
home-timeline-service	2	3	2	2	0.096	0.012	0.182	0.074	0.000	2.0	2.0	15.38	15.57
media-service	2	0	1	0	0.036	0.000	0.121	0.061	0.000	2.0	2.0	16.06	14.83
text-service	2	2	1	2	0.064	0.036	0.121	0.061	0.000	2.0	2.0	15.31	14.39
unique-id-service	2	0	1	0	0.036	0.000	0.121	0.061	0.000	2.0	2.0	14.15	14.64
user-timeline-service	2	1	2	1	0.079	0.003	0.182	0.066	0.000	2.0	2.0	15.73	14.88
compose-post-service	1	10	1	7	0.302	0.053	0.091	0.062	0.000	1.0	1.0	15.55	15.56

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

TABLE IV
HOTEL RESERVATION: GRAPH PREDICTORS PER SERVICE WITH ANCESTOR_COUNT AND THE MEASURED SERVICE-MEDIAN OUTCOMES

service	ancestor_ count (services)	descendant_ count (services)	in_degree_ raw (callers)	out_ degree_ raw (callees)	hybrid_ criticality	betweenness	closeness	pagerank	eigenvector	measured_ ancestor_ affected_ kill (services)	measured_ ancestor_ affected_ latency (services)	T_rec_ kill_s (s)	T_rec_ latency_s (s)
geo	2	0	1	0	0.057	0.000	0.190	0.152	0.707	2.0	2.0	22.01	17.69
rate	2	0	1	0	0.057	0.000	0.190	0.152	0.707	2.0	2.0	20.85	16.87
profile	1	0	1	0	0.057	0.000	0.143	0.124	0.002	1.0	1.0	27.00	18.66
recommendation	1	0	1	0	0.057	0.000	0.143	0.107	0.002	1.0	1.0	22.63	25.45
reservation	1	0	1	0	0.057	0.000	0.143	0.127	0.002	1.0	1.0	19.33	16.55
search	1	2	1	2	0.112	0.048	0.143	0.119	0.002	1.0	1.0	22.28	24.74
user	1	0	1	0	0.057	0.000	0.143	0.119	0.002	1.0	1.0	19.69	13.64

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

B. The Confirmatory Test of H1

H1 replaced the original fan-out hypothesis after the pilot. On Social Network under kill faults, `ancestor_count` and the measured `ancestor_affected_count` correlate at $\rho = 0.785$ across 11 services, with a bias-corrected and accelerated bootstrap interval of [0.000, 1.000]. The uncorrected permutation p-value, from 10,000 Monte-Carlo permutations, is 0.0080. The exact value, 0.006926, is given in Table V (Deviation 9). After Benjamini-Hochberg correction across the 44 tests of the confirmatory family, the adjusted p-value is 0.3451.

TABLE V
SUPPLEMENTARY: EXACT-ENUMERATION P FOR THE PRIMARY TEST AND ITS BH-FDR RERUN

label	SUPPLEMENTARY - not part of the preregistered analysis
test	Social Network, ancestor_count vs ancestor_affected_count, kill
rho	0.785119
p_raw_locked_monte_carlo	0.007999
p_raw_exact_enumeration	0.006926
distinct_arrangements (arrangements)	9240
p_bh_locked	0.3451
p_bh_with_exact	0.3048
significant_locked	no
significant_with_exact	no
verdict	unchanged

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

At $q = 0.05$, not one of the 44 confirmatory tests is significant. The primary test has the smallest uncorrected p-value in the family, and its adjusted value is about 6.9 times the threshold. Table VI gives the primary test in each family and architecture, and Table VII summarises both families. H1 is not confirmed. The point estimate is large and positive, the correction fixed in advance rejects it, and the confidence interval runs from no association to a perfect one.

TABLE VI
THE PRIMARY PREDICTOR-OUTCOME TEST IN EACH FAMILY AND ARCHITECTURE

family	architecture	predictor	outcome	n_ services (services)	rho	ci_lo	ci_hi	ci_ method	p_raw	p_method	p_bh_ adjusted	significant_ q05	role
confirmatory (kill)	Social Network	ancestor_ count	ancestor_ affected_ count	11	0.785	0.000	1.000	BCa	0.0080	10000 random permutations	0.3451	no	confirmatory
confirmatory (kill)	Hotel Reservation	ancestor_ count	ancestor_ affected_ count	7	1.000	1.000	1.000	percentile	0.0476	exact enumeration of 5040 permutations	0.3451	no	DIRECTIONAL ONLY (saturated)
replication (latency)	Social Network	ancestor_ count	ancestor_ affected_ count	11	0.785	0.000	1.000	BCa	0.0076	10000 random permutations	0.1848	no	replication
replication (latency)	Hotel Reservation	ancestor_ count	ancestor_ affected_ count	7	1.000	1.000	1.000	percentile	0.0476	exact enumeration of 5040 permutations	0.2328	no	DIRECTIONAL ONLY (saturated)

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

TABLE VII
FAMILY SIZES AND HOW MANY TESTS SURVIVE BH-FDR AT Q=0.05

family	tests (tests)	tests_with_defined_p (tests)	significant_after_bh_q05 (tests)	smallest_raw_p	its_bh_adjusted_p	smallest_p_test
confirmatory (kill)	44	44	0	0.0080	0.3451	Social Network / ancestor_count vs ancestor_affected_count
replication (latency)	44	44	0	0.0076	0.1848	Social Network / ancestor_count vs ancestor_affected_count

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

Two features of the data qualify that estimate further, and they matter more than the p-value.

The first is saturation. If every service's measured blast radius equalled its graph ancestor count, a correlation of 1.0 would follow by construction and would test nothing. On Hotel Reservation that is exactly what happened: each service's measured outcome matched its graph predictor in every run, under both fault types and at both run and service level. Hotel Reservation is therefore tautological at the unit of analysis, and every Hotel Reservation result is reported as directional only. It can neither confirm the hypothesis nor disconfirm it.

Social Network is not fully saturated. Its run-level saturation is 0.909 under kill faults, which is why ρ is 0.785 and not 1.0. Figure 6 shows the pattern. Ten of the eleven services lie exactly on the line $y = x$. One departs from it: `user-service`, which has 4 graph ancestors and registers 2 degraded ancestors in all five of its kill repetitions. The empirical content of the Social Network correlation, as distinct from its definitional content, rests on that one service.

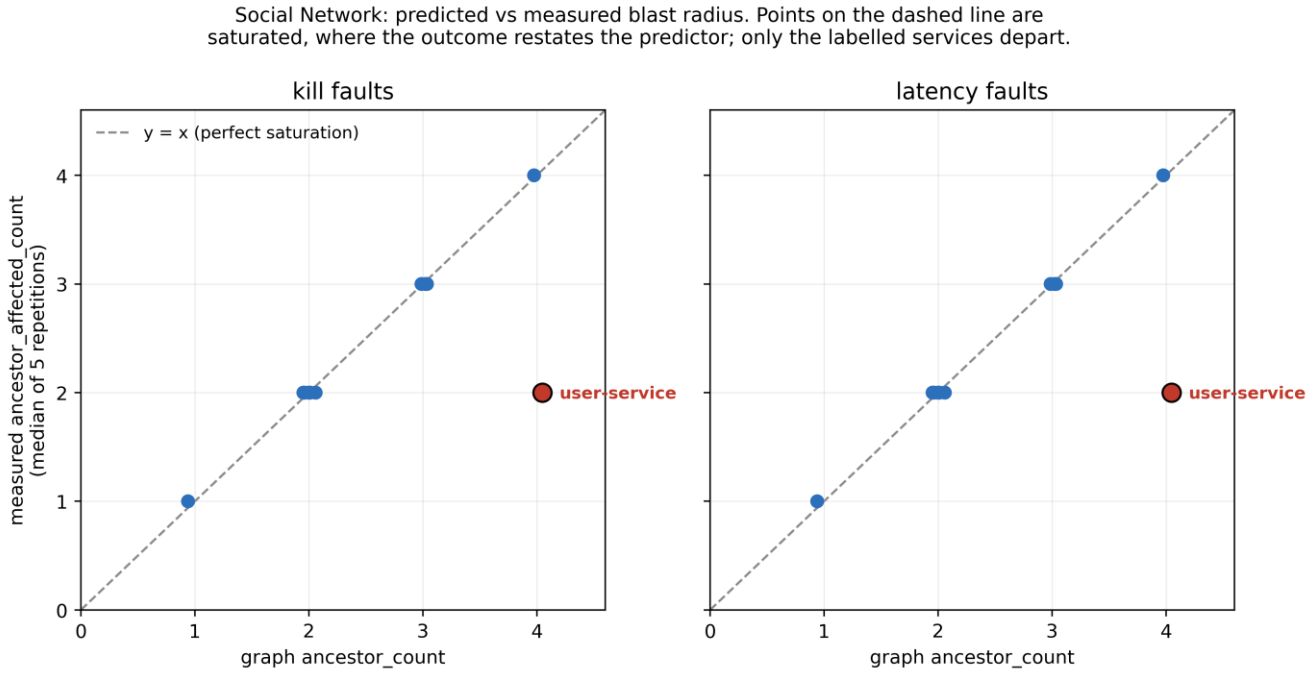


Fig. 6. ancestor_count vs measured ancestor_affected_count per SN service, with y=x and the departing services labelled.

A leave-one-out recomputation repeats the correlation eleven times, withholding one service each time (Table VIII, Fig. 7). The check was designed after the confirmatory results were known, so it is post hoc and carries no inferential weight. Across the eleven recomputations ρ ranges from 0.696 to 1.000. Withholding `user-service` gives ρ = 1.000, a change of +0.215 that is 2.4 times the next largest swing, −0.089 for `compose-post-service`. Only one exclusion moves the correlation much, and it is the one that removes the single service that stops the outcome from restating the predictor.

TABLE VIII
POST-HOC: SOCIAL NETWORK PRIMARY RHO RECOMPUTED WITH EACH SERVICE EXCLUDED

label	fault_type	service_excluded	n_services (services)	rho	p_raw_nominal_uncorrected	delta_rho_vs_full_set	p_method
POST-HOC robustness check - NOT part of the preregistered family	kill	NONE (full set, reference)	11	0.785119	0.006883		2,000,000 random permutations (seed 20260928)

label	fault_type	service_excluded	n_services (services)	rho	p_raw_nominal_ uncorrected	delta_rho_vs_full_ set	p_method
POST-HOC robustness check - NOT part of the preregistered family	kill	compose-post-service	10	0.696454	0.040476	-0.088665	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	home-timeline-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	media-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	post-storage-service	10	0.742746	0.016667	-0.042373	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	social-graph-service	10	0.791490	0.003175	+0.006371	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	text-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	unique-id-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	url-shorten-service	10	0.791490	0.003175	+0.006371	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	user-mention-service	10	0.791490	0.003175	+0.006371	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	user-service	10	1.000000	0.000198	+0.214881	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	kill	user-timeline-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	NONE (full set, reference)	11	0.785119	0.006883		2,000,000 random permutations (seed 20260928)
POST-HOC robustness check - NOT part of the preregistered family	latency	compose-post-service	10	0.696454	0.040476	-0.088665	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	home-timeline-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	media-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	post-storage-service	10	0.742746	0.016667	-0.042373	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	social-graph-service	10	0.791490	0.003175	+0.006371	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	text-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations
POST-HOC robustness check -	latency	unique-id-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800

label	fault_type	service_excluded	n_services (services)	rho	p_raw_nominal_ uncorrected	delta_rho_vs_full_ set	p_method
NOT part of the preregistered family							permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	url-shorten-service	10	0.791490	0.003175	+0.006371	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	user-mention-service	10	0.791490	0.003175	+0.006371	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	user-service	10	1.000000	0.000198	+0.214881	exact enumeration of 3,628,800 permutations
POST-HOC robustness check - NOT part of the preregistered family	latency	user-timeline-service	10	0.763252	0.016270	-0.021867	exact enumeration of 3,628,800 permutations

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

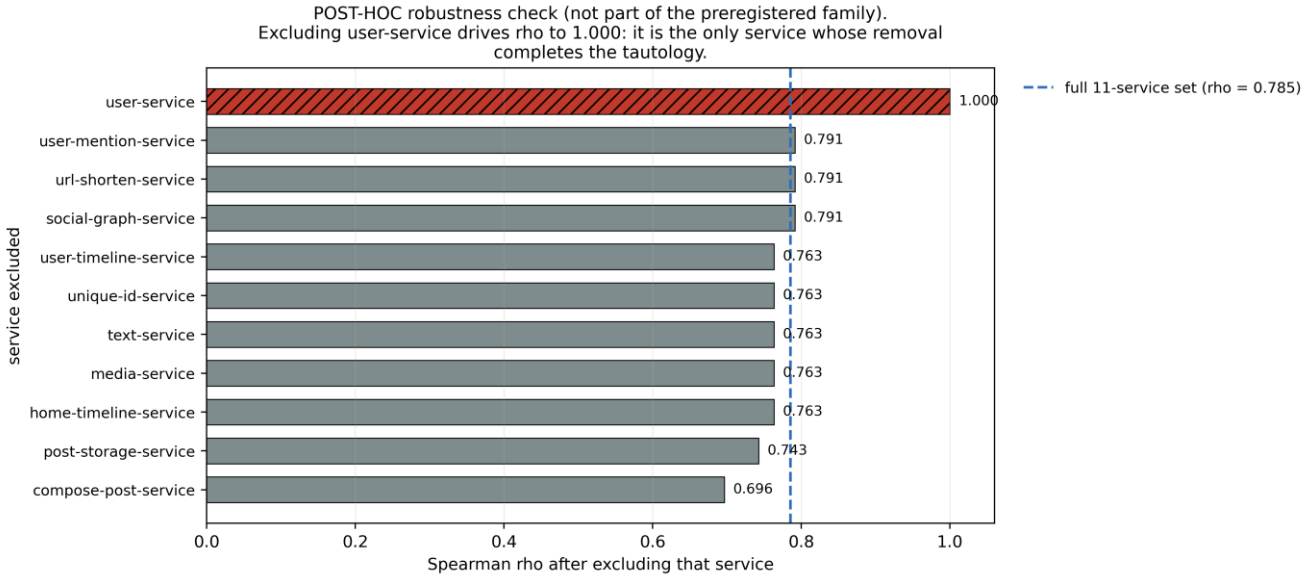


Fig. 7. Post-hoc: primary rho recomputed with each SN service excluded, kill faults.

C. The Hybrid Metric and H2

The metric this work set out to evaluate combines inbound centrality with a fan-out-scaled outbound term, on the assumption that a service calling many others matters because its failure breaks many downstream links. H2 (H1b in the preregistration) expected this premise to point the wrong way.

Table IX gives all four architecture-by-fault cells. They are not independent. The outcome vectors coincide across fault types, so the cells contain 2 distinct comparisons, one per architecture. The correlation between `hybrid_criticality` and `ancestor_affected_count` is negative in every cell: $\rho = -0.144$ on Social Network under both kill and latency faults, and $\rho = -0.258$ on Hotel Reservation. After correction, no cell's correlation can be told apart from zero. The adjusted p-values are 0.9925 and 0.9152 on Social Network and 1.0000 on Hotel Reservation, and the Social Network interval runs from −0.810 to 0.698.

TABLE IX
HYBRID_CRITICALITY VS ANCESTOR_AFFECTED_COUNT IN ALL FOUR ARCHITECTURE-BY-FAULT CELLS (THE PREREGISTRATION'S H1B)

family	fault_type	architecture	n_services (services)	rho	ci_lo	ci_hi	p_raw	p_bh_ adjusted	rho_le_zero_ as_ preregistered	distinguishable_ from_zero_ after_fdr
confirmatory	kill	Social Network	11	-0.144	-0.810	0.698	0.6688	0.9925	yes	no
confirmatory	kill	Hotel Reservation	7	-0.258	-0.730	-0.167	1.0000	1.0000	yes	no
replication	latency	Social Network	11	-0.144	-0.805	0.713	0.6656	0.9152	yes	no
replication	latency	Hotel Reservation	7	-0.258	-0.750	-0.167	1.0000	1.0000	yes	no

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

So the hybrid metric has no detectable association with measured blast radius, even though every point estimate is negative. The direction stated by H2 appears consistently, but its size cannot be separated from zero, and no negative relationship is claimed.

D. The Motivating Case

The metric was first motivated by `compose-post-service`, the composition service of Social Network, which calls 7 services directly and has 10 transitive descendants. Because of that fan-out, the hybrid metric ranks it first of the 11 analysed services, with a score of 0.302. By transitive callers it ranks last, having a single graph ancestor. Figure 8 shows the full rank inversion across the eleven services.

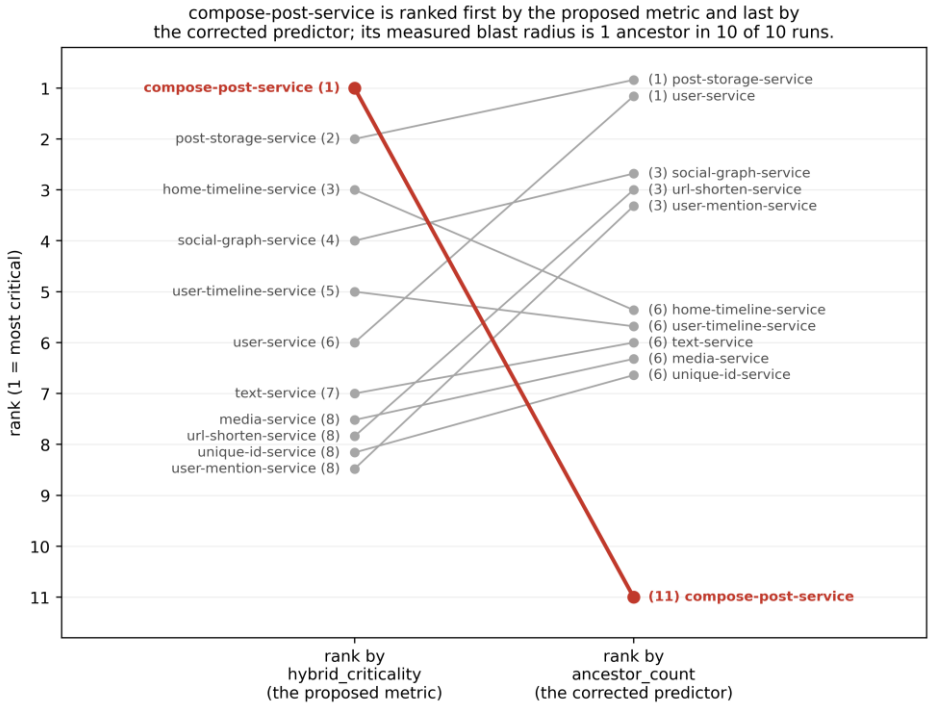


Fig. 8. Rank of all 11 SN services by hybrid_criticality vs by ancestor_count, compose-post-service highlighted.

Measurement sides with the caller-based ranking. Faulting `compose-post-service` left exactly one degraded ancestor in every run. In each of its 5 kill repetitions the minimum and maximum were both 1, and the latency runs gave the same result. A post hoc breakdown shows that in all 10 runs that ancestor was `nginx-web-server`, the gateway. The third kill repetition also recorded two degraded descendants, the only such run in the campaign (Section V-F). On the preregistered outcome, the service the original metric rates most critical is the lowest in its application. That outcome counts degraded services; it does not measure requests or functionality lost.

This is a worked case and not a general finding. It concerns one service in one application, and the test across all eleven services, which could have generalised it, is the null result of Section V-B.

E. Replication under a Second Fault Type

The replication family repeats all 44 tests on latency faults. As in the confirmatory family, no test is significant after correction. The smallest uncorrected p-value is 0.0076, with an adjusted value of 0.1848.

The agreement between the families comes with a caveat. Under kill and latency faults, the service medians of

`ancestor_affected_count` are the same in both architectures, and at run level the match is nearly as close: for 17 of the 18 services, the five per-run counts are the same under both fault types. On the primary outcome, then, the latency family repeats the kill family's correlations on identical data, so the two cannot disagree. The replication adds no independent evidence about the primary hypothesis.

Taken as a finding in its own right, this says that which services degrade when a given service fails hardly depends on how it fails. A process that disappears and a process that slows down reach the same neighbours. Recovery is a different matter. The correlation between median recovery times under kill and under latency faults shows moderate agreement on Social Network (0.384) and strong agreement on Hotel Reservation (0.857, n = 7). For recovery time the replication remains informative, since there the two fault types give different values (Section V-I).

F. Direction of Propagation

Across the campaign, degradation travelled to callers and not to callees. No descendant of the faulted service degraded in 179 of 180 runs, and the service-level median, which is the unit of analysis, is zero for each service under both fault types. The single exception recorded two degraded descendants, in a Social Network `compose-post-service` kill repetition. Services unrelated to the faulted node behave the same way, with 179 of 180 runs at zero and one exception.

This is the firmest structural result in the study, and it cannot be expressed as a correlation. Neither outcome varies between services, so 88 of the 154 exploratory tests attempted have no defined value. A constant outcome does not give a weak correlation. It gives none. These tests are reported as undefined, with the reason, in the same way as the two degenerate predictors, and are not entered as zeros.

G. Exploratory Findings

Of the 66 testable exploratory tests, 2 survive correction within that family. Both concern the size of latency inflation under latency faults on Social Network. Δp95 correlates with `hybrid_criticality` at ρ = 0.949 and with plain `degree` at ρ = 0.953, both with an adjusted p of 0.0033.

These results are exploratory and carry no confirmatory weight. Their outcome is not the one H2 concerns. How much latency inflates is a different quantity from how many ancestors degrade, and doing well on the first says nothing about the second, so this does not rescue the hybrid metric. Plain degree centrality also does at least as well as the hybrid metric on this outcome, which gives no reason to prefer the fan-out correction over the simpler measure.

H. What the Design Could Have Detected

Table X reproduces the preregistered power analysis against the realised analysis sets. Both sets have the registered size and the registered number of distinct predictor levels, and the ceiling correlations come out exactly as preregistered.

TABLE X
PREREGISTERED POWER TABLE REPRODUCED FROM THE ACTUAL ANALYSIS SETS

architecture	**Social Network**	**Hotel Reservation**
n_services_actual (services)	11	7
n_services_ preregistered (services)	11	7
n_matches	yes	yes
ancestor_count_levels_ actual (levels)	4	2
ancestor_count_levels_ preregistered (levels)	4	2
levels_match	yes	yes
ancestor_count_values	1,2,2,2,2,2,3,3,3,4,4	1,1,1,1,1,2,2
ceiling_rho_untied_ outcome	0.941	0.791
ceiling_rho_ preregistered	0.941	0.791
ceiling_matches_prereg	yes	yes
ceiling_p_exact	0.0002	0.0952
ceiling_p_asymptotic_ NOT_FOR_ INFERENCE	0.0000	0.0343
prereg_section_7_2_ states		0.034
role	confirmatory	held-out, DIRECTIONAL ONLY

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

Those ceilings limit what the study could ever show. `ancestor_count` takes only 4 distinct values across the 11 Social Network services and 2 across the 7 Hotel Reservation services, and the ties cap the attainable correlation at 0.941 and 0.791 respectively. For Hotel Reservation the effect is severe. Under the exact permutation test the plan mandates, the best attainable p-value is 0.0952, above the conventional threshold before any correction. The preregistration quoted p = 0.034 for this ceiling, which is the asymptotic value (Deviation 7); the exact value the plan requires is 0.0952. Hotel Reservation could not have reached nominal significance for this predictor whatever outcome was observed. That follows from its tie structure and sample size, not from anything measured.

I. Recovery Time

No run was censored, so under the preregistered rule the Spearman rank test is the primary test for recovery time, and the Cox and log-rank models are secondary and uncorrected (Deviation 6). Table XI gives all three for both predictors. No recovery-time test is significant after correction in either family. The log-rank test is undefined for 10 of the 22 recovery-time tests in each family, including `ancestor_count` on Hotel Reservation, where the predictor has fewer than two distinct terciles. The unrounded estimates are listed in the Appendix.

TABLE XI

RECOVERY TIME: SPEARMAN (PRIMARY, BH-CORRECTED WITHIN FAMILY), COX AND LOG-RANK (SECONDARY, UNCORRECTED) FOR ANCESTOR_COUNT AND HYBRID_CRITICALITY. COX HAZARD RATIOS ARE PER ONE-UNIT INCREASE OF THE PREDICTOR, AT RUN LEVEL WITH SES CLUSTERED BY SERVICE AND AT SERVICE LEVEL; HYBRID_CRITICALITY SPANS LESS THAN ONE UNIT, SO ITS HAZARD RATIOS EXTRAPOLATE BEYOND THE OBSERVED RANGE. LOG-RANK IS UNDEFINED WHERE THE PREDICTOR HAS FEWER THAN TWO DISTINCT TERCILES

family	architecture	predictor	spearman_ rho	p_raw	p_bh_ adjusted	cox_run_ hr (per unit of predictor)	cox_run_ ci_95 (per unit of predictor)	cox_run_p	cox_ service_hr (per unit of predictor)	cox_ service_ci_ 95 (per unit of predictor)	cox_ service_p	logrank_p
confirmatory (kill)	Social Network	ancestor_ count	-0.356	0.2807	0.9264	1.173	0.876 to 1.570	0.2836	1.479	0.688 to 3.181	0.3163	0.5154
confirmatory (kill)	Social Network	hybrid_ criticality	0.098	0.7699	0.9925	0.010	0.002 to 0.043	<0.0001	1.408	5.49e-04 to 3.61e+03	0.9319	0.2099
confirmatory (kill)	Hotel Reservation	ancestor_ count	-0.158	0.8571	0.9925	1.522	1.067 to 2.171	0.0204	2.179	0.302 to 15.722	0.4399	undefined
confirmatory (kill)	Hotel Reservation	hybrid_ criticality	0.204	0.8571	0.9925	0.147	8.91e-04 to 24.134	0.4609	not estimable	not estimable	not estimable	undefined
replication (latency)	Social Network	ancestor_ count	-0.060	0.8647	1.0000	0.998	0.816 to 1.221	0.9859	1.184	0.627 to 2.236	0.6018	0.5561
replication (latency)	Social Network	hybrid_ criticality	0.767	0.0084	0.1848	0.045	0.002 to 1.163	0.0616	2.49e-04	8.19e-10 to 75.493	0.1975	0.1099
replication (latency)	Hotel Reservation	ancestor_ count	-0.158	0.8571	1.0000	0.882	0.578 to 1.346	0.5609	2.179	0.302 to 15.722	0.4399	undefined
replication (latency)	Hotel Reservation	hybrid_ criticality	0.408	0.5714	0.8111	6.26e-04	1.07e-06 to 0.368	0.0234	not estimable	not estimable	not estimable	undefined

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

Not estimable: the Cox fit did not converge or showed separation because the predictor takes few distinct values; raw estimates are in the supplementary file.

J. Post Hoc Weight Sensitivity

The confirmatory results were already known when this analysis was specified and run, and it belongs to no preregistered family. It asks how much the hybrid metric's association with the outcome depends on its weights. The metric was recomputed for all 231 weight triplets on the preregistered grid, with step 0.05 and weights summing to one, using the same code, the same service medians and the same permutation tests. Each set of 231 tests was corrected with the Benjamini-Hochberg procedure within that set. No triplet was selected or adopted, and every other result in this paper uses the reported weights. Table XII and Fig. 9 give the full distribution.

TABLE XII

POST HOC WEIGHT SENSITIVITY OF THE HYBRID METRIC OVER 231 WEIGHT TRIPLETS: DISTRIBUTION OF SPEARMAN RHO, COUNTS OF POSITIVE AND SIGNIFICANT CORRELATIONS (BH WITHIN EACH FAMILY OF 231), AND THE VALUES AT THE REPORTED WEIGHTS

architecture	fault_type	outcome	triplets_ defined	rho_min	rho_median	rho_max	rho_positive	p_raw_ below_05	significant_ bh_231	rho_at_ 0.4/0.4/0.2	p_at_ 0.4/0.4/0.2
Social Network	kill	ancestor_ affected_ count	231	-0.544	-0.167	0.337	28	0	0	-0.144	0.6695
Social Network	latency	ancestor_ affected_ count	231	-0.544	-0.167	0.337	28	0	0	-0.144	0.6695
Hotel Reservation	kill	ancestor_ affected_ count	230	-0.258	-0.258	-0.258	0	0	0	-0.258	1.0000
Hotel Reservation	latency	ancestor_ affected_ count	230	-0.258	-0.258	-0.258	0	0	0	-0.258	1.0000
Social Network	kill	delta_error_ rate	231	0.229	0.753	0.832	231	145	128	0.786	0.0064
Hotel Reservation	kill	delta_error_ rate	230	-0.204	-0.204	-0.204	0	0	0	-0.204	0.8571

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

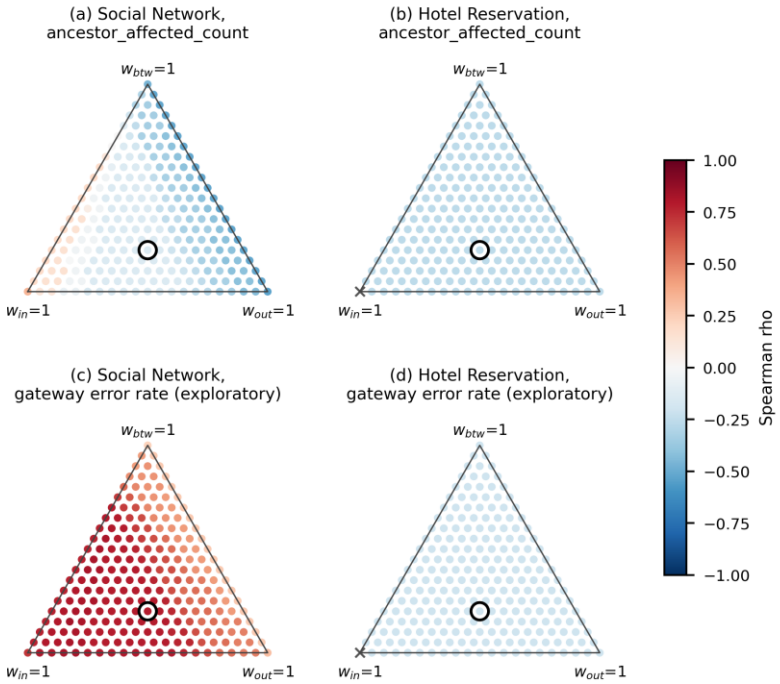


Fig. 9. Post hoc weight sensitivity: Spearman rho of the hybrid metric against each outcome for all 231 weight triplets (kill faults; latency faults give identical ancestor_affected_count results); circled point: the reported weights 0.4/0.4/0.2; x: metric constant.

Against `ancestor_affected_count` on Social Network, ρ ranged from −0.544 to 0.337, with a median of −0.167. It was positive for 28 of the 231 triplets. No triplet reached an uncorrected p below 0.05, the smallest being 0.0952, and none survived correction. Kill and latency faults gave identical results, because the service medians coincide. On Hotel Reservation every triplet with a defined correlation gave ρ = −0.258, 230 in all, because the in-degree-only weighting leaves the metric constant there. The reported weights gave ρ = −0.144 on Social Network, and 47 triplets gave a larger value. The null result for H2 therefore does not depend on the inherited weights: no weighting on the grid produced a significant association with degraded callers in either direction.

Against the gateway's error rate under kill faults, an exploratory outcome, the picture on Social Network differs. ρ was positive for all 231 triplets, from 0.229 to 0.832 with a median of 0.753. Of these, 145 had uncorrected p-values under 0.05, and 128 remained significant after correction within the 231; the p-values come from 10,000 random permutations, because this outcome has too many distinct orderings to enumerate. At the reported weights ρ was 0.786. The largest value, 0.832, is the best of 231 tries on one dataset and not a finding. On Hotel Reservation every defined triplet gave ρ = −0.204 and none was significant. At the reported weights, the preregistered exploratory test of this association did not survive the correction applied across its exploratory family (adjusted p = 0.0913), and Section VI-C shows that this association follows the metric's in-degree term.

K. Verification

Two checks beyond the preregistered analysis are reported as supplementary. One is the post hoc leave-one-out recomputation of Section V-B. The other is an independent reimplementation of the primary statistic, written without reference to the analysis code and reading only the persisted spans and the dependency graph. It reproduces the reported correlation and agrees with the stored per-run outcomes on which services were affected, not only how many, in all 110 Social Network runs. That shows the analysis code is faithful to its data. Whether the definitions of Sections IV-G and IV-H are the right ones is a question no recomputation can settle.

VI. DISCUSSION

A. Why Fan-Out Points the Wrong Way

The direction of propagation is the result that stands with the least qualification. In 179 of 180 runs no descendant of the faulted service degraded, and the service-level median of degraded descendants was zero for every service under both fault types. The set of services that degraded also barely depended on how the fault was delivered. For 17 of the 18 services, the five per-run counts were identical whether the service was killed or slowed down.

The most plausible reading is that a caller suffers while it waits on a dependency that has died or slowed, whereas a callee simply receives fewer requests and has nothing to wait for. The measurements fit that reading, but they do not test it. The degradation rule asks only whether a service's own latency or error rate rose past a threshold, and it never observes waiting directly, so the mechanism remains an interpretation.

If the reading is right, a fan-out ranking scores the wrong end of each edge. The composition service in Social Network is the clearest case. It calls 7 services and the fan-out metric ranks it first, yet when it was faulted it had exactly one degraded caller in each of its 10 runs, and in a post hoc breakdown that caller was always the gateway. Across all services, the metric's correlation with degraded callers was negative in every cell and separable from zero in none (Section V-C), and no weighting on the preregistered grid changed that (Section V-J). That is an absence of support for the fan-out premise. The data cannot show a negative relationship.

B. A Predictor That Sits Close to Its Outcome

The caller count correlates with degraded callers partly by construction, since the outcome counts a subset of the predictor. Whenever every ancestor of a faulted service degrades, the two are equal. That happened for every Hotel Reservation service and for ten of the eleven Social Network services.

A post hoc breakdown of the degraded ancestors makes the point sharper. In all 180 runs the gateway was one of them, and in both applications every analysable service has the gateway among its graph ancestors. Every measured ancestor count therefore includes one unit that is the same for every service. What remains to be explained comes from the intermediate callers, and in Social Network only `user-service` departs from its graph count in a consistent way. It has 4 graph ancestors and registered 2 degraded ancestors in all five kill runs. Its two ancestors that degraded sit on request paths the workload exercises, and its other two showed no degradation in any run. That explanation was formed after seeing the data and has not been tested.

The leave-one-out check, also post hoc, shows what follows from this. With `user-service` withheld, the Social Network correlation becomes 1.000. A reader can fairly treat the Social Network result as resting on one service's worth of independent information. One lesson may extend past this study in a narrow sense: any structural predictor tested against a structurally defined outcome needs a saturation measure reported next to the correlation, or the correlation cannot be interpreted.

C. What Users Would See

The preregistered outcome counts degraded services. It says nothing about how many requests fail. To get closer to what a user experiences, the gateway's end-to-end error rate under kill faults was examined after the fact. This analysis is exploratory and descriptive, and none of its tests enters a preregistered family.

Gateway error followed the request mix. For each faulted service, the Locust task weights of the request types whose traces pass through it were summed and divided by the total weight. The rank correlation between that share and the median gateway error was 0.788 on Social Network and 0.964 on Hotel Reservation. These are descriptive values with no p-value attached. In Hotel Reservation, killing `reservation`, which sits on request types carrying 0.727 of the weight, gave a median gateway error rate of 0.7340. Killing `recommendation`, with a share of 0.091, gave 0.0860. The mix does not explain everything. The Social Network services reached only through the compose request all share a weight of 0.300, and their median error rates still range from 0.1692 to 0.2571.

The hybrid metric's exploratory correlation with gateway error on Social Network was 0.786, not significant after correction within its family (adjusted $p = 0.0913$). Splitting the metric into its weighted terms shows where that association comes from. The in-degree term alone has a rank correlation of 0.715 with gateway error, the fan-out term 0.310 and the betweenness term 0.229. The association the metric shows here follows the count of direct callers more closely than the fan-out correction, so it lends that correction no support.

D. Lessons about Measurement

Two measurement choices mattered more than expected at the planning stage. The first is where latency is attributed. Charging each trace's end-to-end latency to every service on the trace makes services on a shared path indistinguishable. In the pilot comparison of Section IV-G that shortcut flagged five non-faulted services where per-service attribution flagged one. The correction was in place before the campaign began, so what an uncorrected campaign would have concluded is unknown. The pilot shows how it would have behaved.

The second is that per-service attribution is only as good as the instrumentation behind it. Social Network's spans carry no span-kind tag, so its per-service latency drew on all of a service's spans, and 49.7% of those were client spans. A post hoc replay limited to server spans altered 2 of the 110 run outcomes and none of the service-level medians. The change was small, but that was fortunate. A study that depends on span semantics should check them before collecting data, and this one did not.

E. Implications for Practitioners

For a team that has to rank services before running any experiment, the data favour looking up the graph at callers over looking down it at callees. That recommendation rests on the propagation pattern and on the lack of support for fan-out. The preregistered test of the caller ranking itself did not reach significance, so no claim is made that it predicts failure impact. In both applications, the post hoc breakdown of Section VI-B found the gateway in every caller count, and in the exploratory analysis of Section VI-C the user-visible error rate tracked how much traffic passed through the failed service. A ranking that weights callers by the traffic they carry may therefore be worth testing, as a new preregistered hypothesis. It was not tested here.

The applications are benchmarks, and containment mechanisms such as retries with back-off or circuit breakers were not configured or studied, although they could change how far a failure travels. Larger applications would also lift the ceiling that eleven services place on any correlation, and a direct check that injected faults have been removed would close the gap described in Section VII-E.

VII. THREATS TO VALIDITY

A. Sample Size and the Ceiling on Attainable Correlation

The unit of analysis is the service, so the sample size is 11 for Social Network and 7 for Hotel Reservation. Those are the sizes the applications have. They were registered in advance, and the realised sets match them exactly. Repetition makes each service's measured outcome more precise, which is why 5 repetitions per cell were used, but it adds no points to a correlation. A study designed this way is bounded by how many services the system has.

Ties tighten the bound. `ancestor_count` takes 4 distinct values on Social Network and 2 on Hotel Reservation, and ties cap Spearman's ρ whatever is measured. The best attainable correlations are 0.941 and 0.791. The confirmatory null should be read with that in mind. These ceilings were registered before data collection, so they are a stated property of the design and not an explanation found afterwards.

B. Hotel Reservation Could Not Have Reached Significance

For Hotel Reservation the problem goes beyond low power. Under the exact permutation test the plan mandates, a perfect result with the observed ties, $\rho = 1.0$, yields $p = 0.0952$. That is above the conventional 0.05 threshold before any correction for multiplicity. No outcome that could have been measured on this application would have reached nominal significance for this predictor.

The application is also saturated. Each service's measured count matched its graph count in every run, so a correlation computed here only restates how the outcome was built. Every Hotel Reservation result is reported as directional only, and it should be read neither as replicating the Social Network result nor as failing to. Its value here is as a second, very differently shaped topology on which the propagation-direction result of Section V-F also holds.

C. Two Applications from One Benchmark Suite

Both applications come from one benchmark suite, built from one pinned upstream commit. They are research benchmarks. They were chosen because they are instrumented and reproducible, which makes the experiment repeatable, but they do not sample the space of microservice architectures in any principled way. Containment mechanisms such as retries with back-off or circuit breakers were not configured or studied, and they could change how far a failure travels. Nothing here describes production behaviour.

The workload is synthetic as well, fixed at 25 concurrent users with a fixed request mix. Because the dependency graph is rebuilt from observed traffic, a different mix would give a different graph on the same deployment, and both the predictors and the outcomes would move with it (Section VII-F).

D. The Primary Predictor Is Close to Definitional

The most serious threat to the primary result is conceptual. The outcome counts degraded transitive callers and the predictor counts transitive callers, so the two coincide whenever every ancestor degrades. On Hotel Reservation they always coincide. On Social Network they coincide for ten of the eleven services, the exception being `user-service`, which registers 2 of its 4 ancestors in every kill repetition.

The whole gap between $\rho = 0.785$ and a correlation of exactly 1.0 therefore rests on one service. The post hoc leave-one-out check confirms the arithmetic: dropping `user-service` raises ρ to 1.000, a change 2.4 times the size of the next largest. That check was specified after the confirmatory result was known, and it is reported as a description of the data.

Before collection, the open question was whether a predictor this close to its outcome was worth testing at all, and the saturation measure was registered to detect exactly this problem. The test remained meaningful, because saturation was not guaranteed in advance and does not fully hold. A reader may nonetheless treat the Social Network correlation as carrying roughly one service's worth of independent information.

This study cannot say why `user-service` departs. In all five of its kill repetitions the two ancestors that degraded both sit on request paths the workload drives, and its other two graph ancestors showed no degradation in any run. That suggests `ancestor_count` is a ceiling, and that how close the outcome comes to it depends on the request mix. This explanation was formed after seeing the data and has not been tested. Testing it would need a workload-weighted predictor and a new preregistration.

E. Fault Removal Is Inferred, Not Verified

For latency faults, the runner infers that the `tc netem` rule has been removed from the fact that the injecting process has exited. That inference failed at least once, and Section IV-J describes the incident and the safeguards added afterwards. The safeguards limit the damage without ruling it out. The cross-run baseline gate catches a rule that outlives the run that created it, because the contamination then shows up in later runs' fault-free baselines. A rule that leaked and then cleared within a single run would go unnoticed, since that run's own baseline comes before its fault.

Latency-fault recovery times therefore carry an assumption that the current instrumentation cannot check. If a rule persisted into a run's recovery probe, that run's T_{rec} would be inflated and nothing in the pipeline would flag it. Blast-radius counts are less exposed, since they come from the fault window itself. Closing the gap would need a direct query of the container's queueing discipline, which was not implemented.

F. The Dependency Graph Depends on the Workload

Rebuilding the graph from traces gives the calls the system actually makes, which suits this question. It also makes the graph a property of the deployment and the workload together. Production call graphs have been found to change from request to request [11], and even the fixed benchmark workload here shows the effect. Of the 8 edges implied by Hotel Reservation's published architecture, the traces recover 6. Two are never exercised, and one appears that the documentation does not list.

Since `ancestor_count` is computed on the rebuilt graph, a service's predictor value depends on which of its callers the workload happens to drive. Here the `user` service has one observed caller, where the documentation implies two. A different request mix would give it a different predictor value with nothing about the deployed system changed. This limits how far any relationship reported here carries over to other traffic, and it is the same mechanism the post hoc reading of `user-service` in Section VII-D relies on, which is one more reason to treat that reading as untested.

G. Scope of the Measurement

The study measures whether a service degrades, by a preregistered threshold on its own latency or error rate. It does not establish why any individual service degrades. Two fault types were studied, a process kill and a 500 ms network delay. CPU contention was evaluated in the pilot and dropped before the campaign, because its effect was too small to classify reliably, and the preregistration records that as a scope decision. Other failure modes, from resource exhaustion to slow or corrupted responses, are outside what was measured.

H. Social Network Recovery Times Were Not Replayed

For Social Network, per-service latency included every span a service emitted, client spans among them (Section IV-G, Deviation 11). The blast-radius outcomes could be replayed on server-only spans from the stored baseline and fault windows. Recovery times could not, because the recovery probe's windows were not stored, so Social Network recovery times stand as measured.

I. The Baseline Window Includes the Load Ramp-Up

The fault-free baseline is computed over the 60-second window in which the load generator starts, spawns its users and runs (Section IV-E), not over a separate window after a warm-up. A baseline percentile can therefore include start-up effects that a steady-state baseline would not. The same procedure ran in every run, and the baseline-sanity gate, which compares each run's baseline with the rest of the campaign, reported no exceptions. The size of any start-up effect was not measured.

J. Testbed Records

Some testbed values in Table I were queried after the campaign, on 2026-10-20, and are labelled as measured. The Docker client binary predates the campaign, but the host operating system build and the Docker Desktop version could have changed between the campaign and the query, and no record from the campaign days confirms them.

VIII. CONCLUSION

This study ran a preregistered fault-injection campaign of 180 balanced runs on two benchmark applications to ask whether a service's position in its dependency graph indicates how much damage its failure does. The design crossed every analysable service with two fault types and five repetitions, and the analysis was locked before the first run.

The correlational hypothesis did not survive. The number of transitive callers correlated strongly with the number of degraded callers on Social Network, but after correction not one test survived in either family, the 44 confirmatory and the 44 replication tests alike. The fan-out-weighted metric this project began with showed a negative correlation in every cell and could not be separated from zero in any, and a post hoc sweep over 231 weightings found no weighting with a significant association with degraded callers.

What did hold is structural. Failures reached callers and almost never callees: no descendant degraded in 179 of 180 runs, and the service-level median was zero in every condition. This result needs no inference, and it held in both applications and under both fault types.

For practitioners, this points towards ranking services by their callers and away from ranking them by fan-out, with the caveat that this study's own test of the caller ranking did not reach significance. For researchers, the study shows two steps that are easy to skip: attributing latency to each service's own spans, and measuring how far the outcome merely restates a structural predictor before reporting a correlation between them. Every number in this paper can be regenerated from the logged campaign data.

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) for code development support and language editing; the authors performed all experiments and verified all content.

REFERENCES

[1] N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," in *Present and Ulterior Software Engineering (Springer)*, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.

[2] X. Zhou et al., "Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021, doi: 10.1109/TSE.2018.2887384.

[3] L. C. Freeman, "Centrality in Social Networks: Conceptual Clarification," *Social Networks*, vol. 1, no. 3, pp. 215–239, 1978, doi: 10.1016/0378-8733(78)90021-7.

[4] T. Zimmermann and N. Nagappan, "Predicting Defects Using Network Analysis on Dependency Graphs," in *Proc. 30th International Conference on Software Engineering (ICSE '08)*, 2008, pp. 531–540, doi: 10.1145/1368088.1368161.

[5] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The Preregistration Revolution," *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018, doi: 10.1073/pnas.1708274114.

[6] Y. Gan et al., "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," in *Proc. 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, 2019, pp. 3–18, doi: 10.1145/3297858.3304013.

[7] A. Basiri et al., "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.

[8] V. Heorhiadi, S. Rajagopalan, H. Jamjoom, M. K. Reiter, and V. Sekar, "Gremlin: Systematic Resilience Testing of Microservices," in *2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS)*, 2016, pp. 57–66, doi: 10.1109/ICDCS.2016.11.

[9] P. Alvaro, J. Rosen, and J. M. Hellerstein, "Lineage-driven Fault Injection," in *Proc. 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 331–346, doi: 10.1145/2723372.2723711.

[10] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root Cause Localization of Performance Issues in Microservices," in *NOMS 2020 -*

IEEE/IFIP Network Operations and Management Symposium, 2020, pp. 1–9, doi: 10.1109/NOMS47738.2020.9110353.

[11] S. Luo et al., "Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis," in *Proc. ACM Symposium on Cloud Computing (SoCC '21)*, 2021, pp. 412–426, doi: 10.1145/3472883.3487003.

[12] U. Brandes, "A Faster Algorithm for Betweenness Centrality," *The Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001, doi: 10.1080/0022250X.2001.9990249.

[13] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proc. 7th Python in Science Conference (SciPy2008), G. Varoquaux, T. Vaught, J. Millman (Eds.), Pasadena, CA, USA*, 2008, pp. 11–15. [Online]. Available: http://conference.scipy.org/proceedings/SciPy2008/paper_2/

[14] B. H. Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google Technical Report dapper-2010-1, 2010. [Online]. Available: https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/

[15] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995, doi: 10.1111/j.2517-6161.1995.tb02031.x.

APPENDIX

Table XIII lists every deviation from the preregistered plan, with when it arose, what it affected and what happened to the affected runs; Section IV-J summarises them. The full deviation record, with a description of each deviation and of the change made, is in the repository.

TABLE XIII
DEVIATIONS FROM THE PREREGISTERED PLAN: WHEN EACH AROSE, WHAT IT AFFECTED AND WHAT HAPPENED TO THE AFFECTED RUNS

n	date	title	stage	outcome_affected	runs_invalidated (runs)	runs_disposition
1	2026-09-26	Absolute floor added to the latency-degradation threshold	pre-launch rehearsal (before any campaign data existed)	ancestor_affected_count, descendant_affected_count	3	3 rehearsal runs reset and re-collected. Blast radius was recomputable from persisted spans (compose-post-service 1/1/0 -> 1/0/0) but T_rec was not, because recovery-probe windows are not persisted and downstream_ affected was 1-3 in the majority of samples preceding the first all-clear.
2	2026-09-26	Fault targeting resolved by compose label; stack completeness enforced	79 minutes into the campaign	all outcomes for the affected runs	3	reservation|kill|1 (wrong container faulted), search|kill|1 and user|kill|1 (incomplete topology). All 3 reset and re-collected. All 22 Social Network runs to that point were verified clean; the collision cannot occur there because no SN service name is a substring of 'socialnetwork'.
3	2026-09-26	Graph/Jaeger service names are not docker compose service names	28 minutes after resuming; caused a false abort, no data affected	none	0	No data affected. The abort occurred before any Social Network run started; the gate refused to run rather than producing garbage, which is the designed behaviour acting on a wrong premise.
4	2026-09-26	Leaked pumba netem rule contaminated 10 later runs	detected by the pre-analysis gates immediately after run 180	ancestor_affected_count (systematically under-counted), T_rec for the origin run	11	10 contaminated runs plus the origin run reset and re-collected. The origin run was also invalid on its own terms: recorded as censored on p95, but its fault was still active throughout its recovery probe. It was the campaign's only censored observation; the corrected dataset has none.
5	2026-09-27	Two specification gaps resolved by the principal investigator	before the analysis was written	none (analysis specification only)	0	not applicable
6	2026-09-27	Zero censoring promoted Spearman to primary for T_rec	analysis	T_rec inference	0	not applicable
7	2026-09-27	Documentation inconsistency: section 7.2's stated Hotel Reservation p-value is the asymptotic one	analysis (no execution deviation)	none (HR was already directional-only)	0	not applicable

n	date	title	stage	outcome_affected	runs_invalidated (runs)	runs_disposition
8	2026-09-27	Vectorised Spearman/permutation/bootstrap implementation	analysis (implementation only)	none (equivalence tested)	0	not applicable
9	2026-09-28	Primary test's raw p superseded by exact enumeration (Monte-Carlo -> exact)	after the confirmatory analysis was complete and committed	raw and BH-adjusted p-values only; no rho, CI, effect size, outcome or verdict changed	0	not applicable
10	2026-10-06	Preregistered cross-architecture weight-tuning protocol not executed; inherited weights used	in code before the lock; first recorded after the lock and after the campaign data had been analysed	the hybrid_criticality predictor (H1b and every comparator test using it); no outcome measurement	0	not applicable
11	2026-10-06	Social Network per-service latency used all of a service's spans, recorded as server spans	found after the analysis was complete; sensitivity replay run post hoc	Social Network ancestor/descendant/unrelated affected counts and T_rec (measurement scope); replay shows no service-level change in the affected counts	0	No run reset or re-collected. Stored outcomes retained; the replay is a post-hoc sensitivity analysis in separate files.
12	2026-10-06	hotelreservation|rate|latency|1 attempted twice; first attempt's spans persisted without a run record	during the campaign (2026-09-26); first documented 2026-10-06	none	0	Second attempt retained. Every stack was brought down and the Hotel Reservation stack brought back up at 06:31:48 UTC between the two attempts, so the first attempt's fault cannot have carried into the second.

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

Table XIV gives the unrounded recovery-time estimates behind Table XI, including the Cox fits that Table XI shows as not estimable.

TABLE XIV

SUPPLEMENTARY: UNROUNDED RECOVERY-TIME SPEARMAN, COX AND LOG-RANK ESTIMATES AND NOTES FOR ANCESTOR_COUNT AND HYBRID_CRITICALITY, AS STORED BY THE ANALYSIS

family	**confirmatory (kill)**	**confirmatory (kill)**	**confirmatory (kill)**	**confirmatory (kill)**	**replication (latency)**	**replication (latency)**	**replication (latency)**	**replication (latency)**
architecture	Social Network	Social Network	Hotel Reservation	Hotel Reservation	Social Network	Social Network	Hotel Reservation	Hotel Reservation
predictor	ancestor_count	hybrid_criticality	ancestor_count	hybrid_criticality	ancestor_count	hybrid_criticality	ancestor_count	hybrid_criticality
spearman_rho	-0.3556706068979691	0.0979236493186932	-0.1581138830084189	0.2041241452319314	-0.0604881984520355	0.7670685863297637	-0.1581138830084189	0.4082482904638629
p_raw	0.2806719328067193	0.76992300769923	0.8571428571428571	0.8571428571428571	0.8647135286471352	0.0083991600839916	0.8571428571428571	0.5714285714285714
p_bh_adjusted	0.9264216435499308	0.9924812030075189	0.9924812030075189	0.9924812030075189	1.0	0.1847815218478152	1.0	0.8110599078341013
cox_run_hr (per unit of predictor)	1.173037851309082	0.0100722407704209	1.522127406375697	0.1466165285107417	0.9981866526301854	0.0448513219236117	0.8822104533432488	0.0006264670022376
cox_run_ci_lo	0.8762361149192476	0.0023439600893234	1.0672987074699911	0.0008906976629517	0.816074575011345	0.0017290471455332	0.5782679506346109	1.0661580695907393e-06
cox_run_ci_hi	1.5703733014138994	0.0432814682295263	2.1707810803332674	24.134346958207736	1.2209381642299086	1.1634391135558046	1.3459076940611576	0.3681076156401686
cox_run_p	0.2835819622307338	6.356140286204356e-10	0.0203615474898023	0.4609149881083217	0.9859098804815678	0.0616411621150109	0.5608802315068522	0.0233773701594706
cox_run_n	55.0	55.0	35.0	35.0	55.0	55.0	35.0	35.0
cox_run_note	cluster-robust SE by service	cluster-robust SE by service	cluster-robust SE by service	cluster-robust SE by service	cluster-robust SE by service	cluster-robust SE by service	cluster-robust SE by service	cluster-robust SE by service
cox_service_hr (per unit of predictor)	1.4791319950545243	1.4077635598313794	2.178706454731892	0.1240396945295562	1.1843095604404108	0.0002486840617408	2.178706454731892	1.0187728002315452e-05
cox_service_ci_lo	0.6878303603628515	0.0005494820998446	0.3019255396110698	4.3899435441066045e-19	0.627343023937809	8.192006980573117e-10	0.3019255396110698	4.0336630388514354e-23
cox_service_ci_hi	3.1807718659580986	3606.6656965704087	15.721630644446403	3.5047935501678096e+16	2.2357611090445526	75.49280989456926	15.721630644446403	2573090534570.691
cox_service_p	0.316310268753356	0.9319373654016244	0.4399331263054635	0.9189109275184746	0.6018200656917874	0.1975312942885616	0.4399331263054635	0.5739581660696075
cox_service_n	11.0	11.0	7.0	7.0	11.0	11.0	7.0	7.0
cox_service_note	service-level medians, all events observed	service-level medians, all events observed	service-level medians, all events observed	service-level medians, all events observed	service-level medians, all events observed	service-level medians, all events observed	service-level medians, all events observed	service-level medians, all events observed
logrank_chisq	1.3256657345510123	1.5722505440418468			1.1735995488466544	2.5555544812388518		
logrank_p	0.515389235204324	0.2098804809667901			0.556104098675243	0.1099072481834065		
logrank_groups	3.0	2.0	1.0	1.0	3.0	2.0	1.0	1.0
logrank_note	3 tercile groups	2 tercile groups	fewer than 2 distinct terciles (predictor too tied)	fewer than 2 distinct terciles (predictor too tied)	3 tercile groups	2 tercile groups	fewer than 2 distinct terciles (predictor too tied)	fewer than 2 distinct terciles (predictor too tied)

Units are given in parentheses in the column headers; columns without a unit are dimensionless or categorical.

AUTHOR BIOGRAPHIES

[TO FILL: author photograph, Dakshrajsinh Tank]

DAKSHRAJSINH TANK is a B.Tech student with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. [TO FILL: biography]

[TO FILL: author photograph, Badrinath N]

BADRINATH N is an Assistant Professor with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. [TO FILL: biography]